

Informe Laboratorio 1

Sección 1

Eduardo Valenzuela
e-mail: eduardo.valenzuela1@mail.udp.cl

Agosto de 2025

Índice

1. Descripción	2
2. Desarrollo de Actividades	2
2.1. Actividad 1: Cifrado Cesar	2
2.2. Actividad 2: Modo Stealth	4
2.3. Actividad 3: MitM	10
3. Anexo	14

1. Descripción

1. Usted empieza a trabajar en una empresa tecnológica que se jacta de poseer sistemas que permiten identificar filtraciones de información a través de Deep Packet Inspection (DPI). A usted le han encomendado auditar si efectivamente estos sistemas son capaces de detectar las filtraciones a través de tráfico de red. Debido a que el programa ping es ampliamente utilizado desde dentro y hacia fuera de la empresa, su tarea será crear un software que permita replicar tráfico generado por el programa ping con su configuración por defecto, pero con fragmentos de información confidencial. Recuerde que al comparar tráfico real con el generado no debe gatillar alarmas. De todas formas, deberá hacer una prueba de concepto, en la cual se demuestre que al conocer el algoritmo, será fácil determinar el mensaje en claro. Para los pasos 1,2,3 indicar el texto entregado a ChatGPT y validar si el código resultante cumple con lo requerido.

2. Desarrollo de Actividades

En este laboratorio, a través de tres actividades separadas, se permite comprender cómo generar código mediante IA para cifrar una cadena de texto plano utilizando el método César. Posteriormente, se transmite este texto cifrado dentro de paquetes ICMP por medio de la librería de Python Scapy, la cual permite al usuario construir paquetes personalizados. Finalmente, mediante un último código, se capturan estos paquetes dentro de la red y se descifra el mensaje original transmitido mediante fuerza bruta, aplicando la técnica Man In The Middle (MitM).

2.1. Actividad 1: Cifrado Cesar

Para la primera parte de la actividad se debió crear un algoritmo hecho por ChatGPT en este caso utilizado Copilot, este deberá cifrar con la técnica cesar una cadena de texto entregada por el usuario y devolver el texto cifrado en base a un numero de corrimiento de letras.

- Prompt entregado a copilot: Dame un algoritmo de cifrado César que reciba dos parámetros: el string que será cifrado y el valor de corrimiento. El resultado debe guardarse en una variable llamada string cifrado. Para simplificar el script, utiliza una variable que contenga el abecedario y realiza el desplazamiento de caracteres en base a esta cadena.
- Código entregado por la IA:

Listing 1: Código cifrado cesar

```
1 def cifrado_cesar(texto, corrimiento):  
2     # Variable que contiene el abecedario en inglés  
3     abecedario = "ABCDEFGHJKLMNOPQRSTUVWXYZ"  
4     abecedario_minuscula = "abcdefghijklmnopqrstuvwxyz"  
5
```

```
6     string_cifrado = ""
7
8     # Procesar cada caracter del texto
9     for char in texto:
10         if char in abecedario:
11             # Para letras mayúsculas
12             posicion_actual = abecedario.index(char)
13             nueva_posicion = (posicion_actual + corrimiento) % 26
14             string_cifrado += abecedario[nueva_posicion]
15         elif char in abecedario_minuscula:
16             # Para letras minúsculas
17             posicion_actual = abecedario_minuscula.index(char)
18             nueva_posicion = (posicion_actual + corrimiento) % 26
19             string_cifrado += abecedario_minuscula[nueva_posicion]
20         else:
21             # Para caracteres que no son letras (espacios, números,
22             # signos)
23             string_cifrado += char
24
25     return string_cifrado
26
27 # Ejemplo de uso
28 if __name__ == "__main__":
29     # Parámetros de entrada
30     texto_original = input("Ingrese el texto a cifrar: ")
31     corrimiento = int(input("Ingrese el corrimiento (número de posiciones a mover): "))
32
33     # Cifrar el texto
34     string_cifrado = cifrado_cesar(texto_original, corrimiento)
35
36     print(f"Texto original: {texto_original}")
37     print(f"Corrimiento: {corrimiento}")
38     print(f"Texto cifrado: {string_cifrado}")
```

```
● eduardo-valenzuela@eduardo-valenzuela-Modern-14-B5M:~/Documentos/Lab_Cripto/Lab1$ python3 rdo-valenzuela/Documentos/Lab_Cripto/Lab1/cesar.py
Ingrese el texto a cifrar: Criptografia y seguridad en redes
Ingrese el corrimiento (número de posiciones a mover): 9
Texto original: Criptografia y seguridad en redes
Corrimiento: 9
Texto cifrado: Larycxpajorj h bnpdarmjm nw anmnb
```

Figura 1: Resultado consola cesar.py

- Explicación del código:
 - a) Variables: abecedario, abecedario minúscula, string cifrado.
 - b) Para cada carácter del texto original:
 - Si es una letra, encuentra su posición en el abecedario
 - Aplica el corrimiento usando módulo 26 para manejar el "wrap-around"
 - Obtiene la nueva letra de la posición calculada
 - Si no es una letra, la mantiene sin cambios
- Evaluación del código: El código cumple con lo requerido; sin embargo, al no ser lo suficientemente específico en el prompt, no se incluyó la letra "ñ" en el alfabeto. Esta omisión puede provocar problemas si se requiere cifrar palabras en español que contengan dicha letra.

2.2. Actividad 2: Modo Stealth

Con objetivos de laboratorio se quiere ver como funciona este tipo de cifrado en tráfico ICMP. para ello se enviara con ayuda de la librería Scapy un paquete con la información de un texto cifrado, por lo que la idea es colocar cualquier texto y enviarlo a la red a cualquier IP.

- Prompt entregado a copilot: Dame un código en Scapy que envíe un mensaje ICMP con las siguientes condiciones:
 - El payload del paquete debe tener exactamente 48 bytes de longitud.
 - Los primeros 8 bytes del payload deben mantenerse fijos.
 - El resto del payload debe ir desde 0x10 hasta 0x37.
 - El paquete debe mantener su timestamp.
 - El mensaje cifrado debe encontrarse dentro de estos 48 bytes.
 - Es obligatorio que el paquete tenga exactamente este largo: no debe ser ni menor ni mayor.

Listing 2: Codigo de envio paquete ICMP

```

1  #!/usr/bin/env python3
2  from scapy.all import IP, ICMP, Raw, sr1
3  import sys
4  import time
5  import struct
6  from datetime import datetime
7
8  def cifrado_cesar(texto, corrimiento):
9      """
10     Cifra un texto usando el algoritmo de cifrado C sar
11
12     Args:
13         texto (str): El string que ser  cifrado
14         corrimiento (int): La variable de corrimiento
15
16     Returns:
17         str: El string cifrado
18     """
19     abecedario = "ABCDEFGH IJKLMNOPQRSTUVWXYZ"
20     abecedario_minuscula = "abcdefghijklmnopqrstuvwxyz"
21
22     string_cifrado = ""
23
24     for char in texto:
25         if char in abecedario:
26             posicion_actual = abecedario.index(char)
27             nueva_posicion = (posicion_actual + corrimiento) % 26
28             string_cifrado += abecedario[nueva_posicion]
29         elif char in abecedario_minuscula:
30             posicion_actual = abecedario_minuscula.index(char)
31             nueva_posicion = (posicion_actual + corrimiento) % 26
32             string_cifrado += abecedario_minuscula[nueva_posicion]
33         else:
34             string_cifrado += char
35
36     return string_cifrado
37
38 def crear_payload_48_bytes(mensaje, corrimiento, timestamp_actual):
39     """
40     Crea un payload de exactamente 48 bytes con:
41     - Bytes desde 0x10 a 0x37 (40 bytes) conteniendo el mensaje
42       cifrado
43     - Timestamp (8 bytes)
44
45     Args:
46         mensaje (str): Mensaje a cifrar
47         corrimiento (int): Corrimiento para cifrado C sar
48         timestamp_actual (float): Timestamp Unix actual
49
50     Returns:

```

```

50         bytes: Payload de exactamente 48 bytes
51     """
52     # Cifrar el mensaje
53     mensaje_cifrado = cifrado_cesar(mensaje, corrimiento)
54     mensaje_bytes = mensaje_cifrado.encode('utf-8')
55
56     # Crear los 40 bytes base (0x10 a 0x37)
57     payload_base = bytearray(range(0x10, 0x38)) # 40 bytes
58
59     # Insertar el mensaje cifrado en los primeros bytes disponibles
60     # Limitar a máximo 40 bytes para el mensaje
61     mensaje_truncado = mensaje_bytes[:40]
62
63     # Reemplazar los primeros bytes del payload con el mensaje cifrado
64     for i, byte_msg in enumerate(mensaje_truncado):
65         if i < 40:
66             payload_base[i] = byte_msg
67
68     # Si el mensaje es menor a 40 bytes, completar con los valores
69     # originales
70     # (esto mantiene la secuencia 0x10-0x37 en las posiciones no
71     # usadas)
72
73     # Agregar timestamp (8 bytes)
74     timestamp_bytes = struct.pack('!d', timestamp_actual)
75
76     # Payload final de exactamente 48 bytes
77     payload_final = bytes(payload_base) + timestamp_bytes
78
79 def enviar_icmp_cifrado(destino, mensaje, corrimiento):
80     """
81     Env a un paquete ICMP con mensaje cifrado en payload de
82     exactamente 48 bytes
83
84     Args:
85         destino (str): IP de destino
86         mensaje (str): Mensaje a cifrar y enviar
87         corrimiento (int): Corrimiento para cifrado Cesar
88     """
89     # Timestamp actual UTC
90     timestamp_actual = time.time()
91
92     # Crear payload de exactamente 48 bytes
93     payload_48_bytes, mensaje_cifrado = crear_payload_48_bytes(mensaje,
94         corrimiento, timestamp_actual)
95
96     # Verificar que el payload sea exactamente 48 bytes
97     assert len(payload_48_bytes) == 48, f"Error: Payload debe ser exactamente 48 bytes, actual: {len(payload_48_bytes)}"

```

```

97     print(f"Mensaje original: '{mensaje}'")
98     print(f"Mensaje cifrado (Caracter + {corrimiento}): '{mensaje_cifrado}'")
99     print(f"Longitud del mensaje cifrado: {len(mensaje_cifrado)} bytes")
100
101     print(f"Destino: {destino}")
102     print()
103
104     # Mostrar payload detallado
105     print("--- Payload de 48 bytes ---")
106     print(f"Total: {len(payload_48_bytes)} bytes")
107     print(f"Primeros 40 bytes (con mensaje): {payload_48_bytes[:40].hex()}")
108     print(f"Últimos 8 bytes (timestamp): {payload_48_bytes[40:48].hex()}")
109     print(f"Payload completo: {payload_48_bytes.hex()}")
110     print()
111
112     # Mostrar en formato legible
113     print("Contenido de los primeros 40 bytes:")
114     for i in range(0, 40, 8):
115         chunk = payload_48_bytes[i:i+8]
116         hex_str = ''.join([f"{b:02x}" for b in chunk])
117         ascii_str = ''.join([chr(b) if 32 <= b <= 126 else '.' for b in chunk])
118         print(f"{i:02d}-{i+7:02d}: {hex_str[:23]} | {ascii_str}")
119
120     # Crear paquete ICMP
121     ip_layer = IP(dst=destino)
122     icmp_layer = ICMP(type=8, code=0) # Echo Request
123
124     # Construir paquete con payload de exactamente 48 bytes
125     paquete = ip_layer / icmp_layer / Raw(load=payload_48_bytes)
126
127     print("\n--- Información del paquete ---")
128     paquete.show()
129
130     try:
131         print(f"\nEnviando paquete ICMP con mensaje cifrado...")
132         respuesta = sr1(paquete, timeout=3, verbose=0)
133
134         if respuesta:
135             print("Respuesta recibida:")
136             print(f"Origen: {respuesta.src}")
137             print(f"Tipo ICMP: {respuesta[ICMP].type}")
138             print(f"Código ICMP: {respuesta[ICMP].code}")
139
140             if respuesta.haslayer(Raw):
141                 payload_respuesta = respuesta[Raw].load
142                 print(f"Longitud payload respuesta: {len(payload_respuesta)} bytes")
143                 print(f"Payload respuesta: {payload_respuesta.hex()}")

```

```

143         (})}")
144
145     # Intentar descifrar el mensaje de la respuesta
146     if len(payload_respuesta) >= 40:
147         try:
148             # Extraer los primeros 40 bytes
149             mensaje_bytes_respuesta = payload_respuesta
150             [:40]
151
152             # Convertir a string (solo caracteres
153             imprimibles)
154             mensaje_respuesta = ''.join([chr(b) if 32 <= b
155             <= 126 else '' for b in
156             mensaje_bytes_respuesta])
157
158             # Intentar descifrar
159             mensaje_descifrado = cifrado_cesar(
160                 mensaje_respuesta, -corrimiento)
161             print(f"_{Mensaje_en_respuesta:}_{
162                 mensaje_respuesta}")
163             print(f"_{Mensaje_descifrado:}_{
164                 mensaje_descifrado}")
165
166         except Exception as e:
167             print(f"_{Error_al_descifrar_respuesta:}_{e}"
168                 )
169
170     # Extraer timestamp si est presente
171     if len(payload_respuesta) >= 48:
172         try:
173             timestamp_respuesta = struct.unpack('!d',
174                 payload_respuesta[40:48])[0]
175             timestamp_respuesta_utc = datetime.
176                 utcfromtimestamp(timestamp_respuesta)
177             print(f"_{Timestamp_respuesta:}_{
178                 timestamp_respuesta_utc.strftime('%Y-%m-%d
179                 %H:%M:%S')}_{UTC}")
180
181             # Calcular RTT
182             rtt = time.time() - timestamp_respuesta
183             print(f"_{RTT:}_{rtt*1000:.2f}_{ms}")
184         except:
185             print(f"_{No_se_pudo_extraer_timestamp_de_
186                 respuesta}")
187
188     else:
189         print(f"_{No_se_recibi_respuesta_(timeout)}")
190
191     except Exception as e:
192         print(f"_{Error_al_enviar_paquete:}_{e}")
193
194 if __name__ == "__main__":
195     # Configuraci n

```



```

181     destino_default = "8.8.8.8"
182     mensaje_default = "criptografia_y_seguridad_en_redes"
183     corrimiento_default = 9
184
185     # Obtener par metros
186     if len(sys.argv) >= 4:
187         destino = sys.argv[1]
188         mensaje = sys.argv[2]
189         corrimiento = int(sys.argv[3])
190     else:
191         destino = input(f"IP_destino(default: {destino_default}): ").strip() or destino_default
192         mensaje = input(f"Mensaje_a_cifrar(default: {mensaje_default}): ").strip() or mensaje_default
193         corrimiento = input(f"Corrimiento_C_sar(default: {corrimiento_default}): ").strip()
194         corrimiento = int(corrimiento) if corrimiento else corrimiento_default
195
196     # Validar IP
197     try:
198         import ipaddress
199         ipaddress.ip_address(destino)
200     except ValueError:
201         print(f"IP inv lida: {destino}")
202         sys.exit(1)
203
204     # Advertencia sobre longitud del mensaje
205     if len(mensaje) > 40:
206         print(f"Advertencia: El mensaje ser truncado a 40 bytes")
207         print(f"Mensaje original: {len(mensaje)} bytes")
208         print(f"Mensaje truncado: {mensaje[:40]}")
209         print()
210
211     print(f"{'='*60}")
212
213     # Enviar paquete
214     enviar_icmp_cifrado(destino, mensaje, corrimiento)

```

- Explicación del código:

- Variables: Destino default, mensaje default, corrimiento default.
- Valuamos lo que queremos enviar, en caso de no haber nada se toman las variables default.
- Validación de IP y de longitud del mensaje.
- Enviamos el paquete obteniendo el timestap actual y se crea un payload de 48 bytes exactos, evaluamos ciertos criterios y se muestra en consola como se vera

el payload. después construye el paquete ICMP con el payload personalizado y enviamos a la IP esperando una respuesta.

- Resultado adjuntos en el anexo del presente documento.
- Evaluación del código: El código cumple con los requerimientos establecidos; sin embargo, presenta problemas con las importaciones de Scapy y utiliza código legacy que actualmente no se encuentra en uso por la librería en el año 2025. Además agregó mucho código basura tal como mostrar en pantalla la información del paquete que no aporta en el resultado final.

2.3. Actividad 3: MitM

Con objetos de laboratorio se capturó el paquete ICMP enviado en la actividad anterior y será puesto a un descifrado bruto en base a un diccionario de palabras comunes. La idea es evaluar que tan seguro es este tipo de cifrado y lo sencillo que es capturar estos paquetes y ver su contenido original con un simple script generado por la IA.

- Prompt entregado a copilot: Escribe un código que capture el paquete ICMP enviado por el código stealth mode y descifre el mensaje enviado. cabe considerar que se supone que este código no debiese de saber cuál es el corrimiento que se utilizó, es por ello que quiero que pruebes 25 corrimientos(1-26) para poder observar en consola el mensaje. También agrega una función que permita reconocer en base a un diccionario de palabras comunes el mensaje original enviado. para hacer más fácil la captación con tan solo que tenga 1 palabra común ya es candidato a ser el mensaje cifrado.
- Código entregado por la IA:

Listing 3: Código MitM

```

1  #!/usr/bin/env python3
2  from scapy.all import ICMP, IP, Raw, sniff
3  import struct
4  from datetime import datetime
5
6  # Diccionario b sico de palabras comunes en espa ol
7  PALABRAS_COMUNES = {
8      "el", "la", "los", "las", "un", "una", "unos", "unas", "y", "o", "
9      a", "ante", "bajo", "con", "contra",
10     "de", "desde", "en", "entre", "hacia", "hasta", "para", "por", "
11     seg n", "sin", "sobre", "tras",
12     "es", "son", "est ", "est n", "ser", "estar", "haber", "tener",
13     "hacer", "decir", "ir", "ver", "dar",
14     "saber", "querer", "poder", "deber", "poner", "parecer", "quedar",
15     "creer", "hablar", "llevar", "dejar",
16     "seguir", "encontrar", "llegar", "volver", "pasar", "estar", "dar"
17     , "tomar", "conocer", "pensar",
18     "criptografia", "seguridad", "redes", "mensaje", "cifrado", "
19     secreto", "informacion", "datos", "cesar",

```

```

14     "algoritmo", "corrimiento", "descifrar", "texto", "comunicacion"
15 }
16
17 def descifrado_cesar(texto_cifrado, corrimiento):
18     """
19     Descifra un texto usando el algoritmo de cifrado C sar
20
21     Args:
22         texto_cifrado (str): El texto cifrado
23         corrimiento (int): El corrimiento utilizado para descifrar
24
25     Returns:
26         str: El texto descifrado
27     """
28     abecedario = "ABCDEFGH IJKLMNOPQRSTUVWXYZ"
29     abecedario_minuscula = "abcdefghijklmnopqrstuvwxyz"
30
31     string_descifrado = ""
32
33     for char in texto_cifrado:
34         if char in abecedario:
35             posicion_actual = abecedario.index(char)
36             nueva_posicion = (posicion_actual - corrimiento) % 26
37             string_descifrado += abecedario[nueva_posicion]
38         elif char in abecedario_minuscula:
39             posicion_actual = abecedario_minuscula.index(char)
40             nueva_posicion = (posicion_actual - corrimiento) % 26
41             string_descifrado += abecedario_minuscula[nueva_posicion]
42         else:
43             string_descifrado += char
44
45     return string_descifrado
46
47 def analizar_texto(texto):
48     """
49     Analiza un texto para determinar cu ntas palabras conocidas
50     contiene
51
52     Args:
53         texto (str): El texto a analizar
54
55     Returns:
56         int: N mero de palabras conocidas encontradas
57     """
58     # Normalizar texto (quitar signos de puntuaci n y convertir a
59     # min sculas)
60     palabras = texto.lower().replace(".", "").replace(",", "").replace(
61         "!", "").replace("?", "").split()
62
63     # Contar palabras que existen en nuestro diccionario
64     contador = 0
65     for palabra in palabras:

```

```

63         if palabra in PALABRAS_COMUNES:
64             contador += 1
65
66     return contador
67
68 def procesar_paquete(paquete):
69     """
70     Procesa un paquete ICMP para extraer y descifrar el mensaje oculto
71     """
72     # Verificar si es un paquete ICMP Echo Request (tipo 8)
73     if ICMP in paquete and paquete[ICMP].type == 8:
74         print("\n Capturado paquete ICMP Echo Request!")
75         print(f"Origen: {paquete[IP].src}, Destino: {paquete[IP].dst}"
76             )
77
78     # Verificar si tiene payload
79     if Raw in paquete:
80         payload = paquete[Raw].load
81         print(f"Longitud del payload: {len(payload)} bytes")
82
83     # Extraer los primeros 40 bytes (mensaje cifrado)
84     if len(payload) >= 40:
85         mensaje_cifrado_bytes = payload[:40]
86
87         # Convertir a string (solo caracteres imprimibles)
88         mensaje_cifrado = ''.join([chr(b) if 32 <= b <= 126
89             else ' ' for b in mensaje_cifrado_bytes])
90         print(f"Mensaje cifrado extra do: {mensaje_cifrado}'
91             ")
92
93     # Intentar descifrar con diferentes corrimientos
94     print("\n=== Posibles mensajes descifrados ===")
95
96     mejor_corrimiento = 0
97     mejor_puntuacion = 0
98     mejor_mensaje = ""
99
100     for corrimiento in range(1, 26):
101         mensaje_descifrado = descifrado_cesar(
102             mensaje_cifrado, corrimiento)
103         print(f"Corrimiento {corrimiento:2d}: {mensaje_descifrado}")
104
105     # Analizar el texto descifrado
106     puntuacion = analizar_texto(mensaje_descifrado)
107
108     # Actualizar si encontramos un mejor candidato
109     if puntuacion > mejor_puntuacion:
110         mejor_puntuacion = puntuacion
111         mejor_corrimiento = corrimiento
112         mejor_mensaje = mensaje_descifrado

```

```

110         # Mostrar el resultado del analisis
111         if mejor_puntuacion > 0:
112             print("\n=== Mensaje Detectado ===")
113             print(f"Corrimiento detectado: {mejor_corrimiento}")
114             print(f"Mensaje descifrado: {mejor_mensaje}")
115             print(f"Palabras reconocidas: {mejor_puntuacion}")
116         else:
117             print("\nNo se pudo determinar automaticamente el mensaje original.")
118
119         # Extraer timestamp si est presente
120         if len(payload) >= 48:
121             try:
122                 timestamp = struct.unpack('!d', payload[40:48])[0]
123                 timestamp_utc = datetime.datetime.fromtimestamp(timestamp)
124                 print(f"\nTimestamp del mensaje: {timestamp_utc.strftime('%Y-%m-%d %H:%M:%S')} UTC")
125             except:
126                 print("\nNo se pudo extraer el timestamp")
127         else:
128             print("Payload demasiado corto para contener un mensaje cifrado")
129
130         else:
131             print("El paquete no contiene payload")
132
133         print("="*60)
134
135     def iniciar_captura():
136         """
137         Inicia la captura de paquetes ICMP
138         """
139         print("="*60)
140         print("Iniciando captura de paquetes ICMP...")
141         print("Esperando detectar mensajes cifrados con Cesar")
142         print("Se analizar automaticamente el mensaje para detectar el corrimiento correcto")
143         print("Presiona Ctrl+C para detener la captura")
144         print("="*60)
145
146         try:
147             # Capturar paquetes ICMP
148             sniff(filter="icmp", prn=procesar_paquete)
149         except KeyboardInterrupt:
150             print("\nCaptura de paquetes detenida por el usuario")
151
152     if __name__ == "__main__":
153         iniciar_captura()

```

- Explicación del código:
 - Por medio de la función sniff de Scapy y un filtro ICMP busca en la red todo este tipo de paquetes enviados en el momento de ejecución del código y les aplica la función procesar paquete
 - Dentro de la función de procesamiento se verifican el tipo de paquete, si contiene payload y extrae los primeros 40 bytes que son del mensaje.
 - El mensaje extraído es puesto a prueba en un ciclo que prueba 25 corrimientos y evalúa por medio de la función de descifrado y análisis de texto que tanto del diccionario tiene la frase y se van cambiando variables temporales para obtener el mejor resultado.
 - Una vez recorrió todas las combinaciones y tiene al mejor candidato se muestra en consola el mensaje descifrado junto con la cantidad de palabras comunes en la frase
 - Resultado adjuntos en el anexo del presente documento.
- Evaluación del código: El código cumple con los requerimientos establecidos; sin embargo, presento problemas con los objetivos del código, ya que sigue mostrando información no especificada y además ocupa código legacy que muestra en pantalla un deprecated.

Conclusiones y comentarios

En base a las actividades desarrolladas, la inteligencia artificial tiene la capacidad de construir código malicioso que capture paquetes. Sin embargo, actualmente las medidas de seguridad implementadas en el envío de información sensible no representan una vulnerabilidad mayor que una posible página web mal configurada. Además, la IA sin una correcta supervisión presenta muchas falencias al construir este tipo de código, ya que la información que posee está desactualizada y asume muchos aspectos si no se especifican claramente. La actividad resultó exitosa única y exclusivamente por el estudiante que fungió como guía para esta herramienta. Sin la debida construcción de un prompt realmente detallado y completo, no se lograrían grandes resultados debido a su información desactualizada con respecto a las librerías actuales. En conclusión, la IA no es capaz por sí sola de construir código que capture paquetes y los analice sin la debida corrección por parte de un humano.

3. Anexo

Referencias

- [1] Repositorio: https://github.com/3dvvarMvb/Lab_Cripto
- [2] Inteligencia artificial utilizada: <https://github.com/copilot>

```

sudo python3 stealth_mode.py

[sudo] contraseña para eduardo-valenzuela:
IP destino (default: 8.8.8.8):
Mensaje a cifrar (default: criptografia y seguridad en redes):
Corrimiento César (default: 9):
=====
Mensaje original: 'criptografia y seguridad en redes'
Mensaje cifrado (César +9): 'larycxpajorj h bnpdarmjm nw anmnb'
Longitud del mensaje cifrado: 33 bytes
Destino: 8.8.8.8

--- Payload de 48 bytes ---
Total: 48 bytes
Primeros 40 bytes (con mensaje): 6c617279637870616a6f726a206820626e706461726d6a6d206e7720616e6d6e6231323334353637
Ultimos 8 bytes (timestamp): 41da2d2b902531e2
Payload completo: 6c617279637870616a6f726a206820626e706461726d6a6d206e7720616e6d6e623132333435363741da2d2b902531e2

Contenido de los primeros 40 bytes:
00-07: 6c 61 72 79 63 78 70 61 | larycxpa
08-15: 6a 6f 72 6a 20 68 20 62 | jorj h b
16-23: 6e 70 64 61 72 6d 6a 6d | npdarmjm
24-31: 20 6e 77 20 61 6e 6d 6e | nw anmn
32-39: 62 31 32 33 34 35 36 37 | b1234567

```

Figura 2: Resultado consola Stealth Mode.py 1

```

--- Información del paquete ---
###[ IP ]###
version = 4
ihl = None
tos = 0x0
len = None
id = 1
flags =
frag = 0
ttl = 64
proto = icmp
chksum = None
src = 192.168.100.13
dst = 8.8.8.8
\options \
###[ ICMP ]###
type = echo-request
code = 0
chksum = None
id = 0x0
seq = 0x0
unused = ''
###[ Raw ]###
load = 'larycxpajorj h bnpdarmjm nw anmn1234567A\\xda-+\\x90%1\\xe2'

```

Figura 3: Resultado consola Stealth Mode.py 2

```

Enviando paquete ICMP con mensaje cifrado...
✓ Respuesta recibida:
- Origen: 8.8.8.8
- Tipo ICMP: 0
- Código ICMP: 0
- Longitud payload respuesta: 48 bytes
- Payload respuesta: 6c617279637870616a6f726a206820626e706461726d6a6d206e7720616e6d6e623132333435363741da2d2b902531e2
- Mensaje en respuesta: 'larycxpajorj h bnpdarmjm nw anmn1234567'
- Mensaje descifrado: 'criptografia y seguridad en redes1234567'
/home/eduardo-valenzuela/Documentos/Lab_Cripto/Lab1/stealth_mode.py:165: DeprecationWarning: datetime.datetime.utcnow() is deprecated and scheduled for removal in a future vers
ion. Use timezone-aware objects to represent datetimes in UTC: datetime.datetime.fromtimestamp(timestamp, datetime.UTC).
timestamp_respuesta.utc = datetime.datetime.utcnow().timestamp()
- Timestamp respuesta: 2825-08-31 20:19:12 UTC
- RTT: 76.77 ms

```

Figura 4: Resultado consola Stealth Mode.py 3

```

~/Documentos/Lab_Cripto/Lab1 (12:50s)
sudo python3 MitM.py

[sudo] contraseña para eduardo-valenzuela:
=====
Iniciando captura de paquetes ICMP...
Esperando detectar mensajes cifrados con César
Se analizará automáticamente el mensaje para detectar el corrimiento correcto
Presiona Ctrl+C para detener la captura
=====
¡Capturado paquete ICMP Echo Request!
Origen: 192.168.100.13, Destino: 8.8.8.8
Longitud del payload: 48 bytes
Mensaje cifrado extraído: 'larycxpajorj h bnpdarmjm nw anmn1234567'

```

Figura 5: Resultado consola MitM.py 1

```

--- Posibles mensajes descifrados ---
Corrimiento 1: kzxkwozlnq g amczatl1 av zmlna1234567'
Corrimiento 2: jypwvnyhph f zlnbypkhh lu ylk1z1234567'
Corrimiento 3: lxouzumglog e ykmaxcgg kt xkky1234567'
Corrimiento 4: lhwuyllwkn1 d xjlzwlfi js wlj1x1234567'
Corrimiento 5: gwntxskvejme c wkkyvnhh tr vihlw1234567'
Corrimiento 6: fulswrjudl1d b vhlxulgdg hq ughw1234567'
Corrimiento 7: ctkxvlttmc a ugwtkfcr qp lpfuq1234567'
Corrimiento 8: dsjqphsbgjb z tfhvsjebe fo sfoft1234567'
Corrimiento 9: criptografia y seguridad en redes1234567'
Corrimiento 10: bghosnfgzha x rffqgczc dm qcd1234567'
Corrimiento 11: agnrmegdydy w qcespgbyb cl pcbq1234567'
Corrimiento 12: zofnqldoxcfx v pbdrofaxa bk obabp1234567'
Corrimiento 13: ynelplcnbhw u oacqnczw aj nazao1234567'
Corrimiento 14: xndkojbmadv t nzbpndvvy z1 mzyzn1234567'
Corrimiento 15: wlcjnlaluzcu s myaolcxux yh lyxym1234567'
Corrimiento 16: vksimzkyol r lsnkbhw xg kxwcl1234567'
Corrimiento 17: ujahlgvysxas q kwmjavsv wf jwwk1234567'
Corrimiento 18: tlzgxxtwzr p jvklzuru ve lwuj1234567'
Corrimiento 19: shvjfwhvyya q lmkhytct ud huts1234567'
Corrimiento 20: rgxelvgguxp n htvgasps tc gtsth1234567'
Corrimiento 21: qfwdhucufotw m gsulfwrar sb fsrsg1234567'
Corrimiento 22: pwcglttssx l frthwqpa ra erof1234567'
Corrimiento 23: odubfasdrrum k eqgdupmp qz dqpqe1234567'
Corrimiento 24: nctwazrcqtl j dprfctolo py cpopt1234567'
Corrimiento 25: mbszgybqsk l coqebnko ox bonoc1234567'

--- MENSAJE DETECTADO ---
Corrimiento detectado: 9
Mensaje descifrado: 'criptografia y seguridad en redes1234567'
Palabras recomendadas: 4
/home/eduardo-valenzuela/Documentos/Lab_Cripto/Lab1/MitM.py:123: DeprecationWarning: datetime.datetime.utcnow() is deprecated and scheduled for removal in a future version. Use timezone-aware objects to represent datetimes in UTC: datetime.datetime.fromtimestamp(timestamp, datetime.UTC).
timestamp_utc = datetime.datetime.fromtimestamp(timestamp)

```

Figura 6: Resultado consola MitM.py 2